

Cloud, Réseaux Virtuels et Conteneurisation

UE CRV — Sorbonne Université

Naceur.Malouch@lip6.fr

Cours 3 : cgroups, LXC et Docker

Mars 2026

Table des matières

1	Les cgroups (Control Groups)	2
1.1	Définition et rôle	2
1.2	Interface avec le noyau : cgroupfs	2
1.3	Contrôleurs disponibles	2
1.4	Exemple : création d'un cgroup	3
2	Logiciels de conteneurisation	3
2.1	LXC – Linux Containers	3
2.2	Docker	4
2.2.1	Architecture Docker	4
3	Récapitulatif : support noyau pour la conteneurisation	4
4	Sources	4

Les cgroups (Control Groups)

Définition et rôle

En plus des namespaces (isolation), les logiciels de conteneurisation utilisent un autre support du noyau : les **cgroups** (*control groups*).

cgroups – Définition

Les cgroups permettent le **contrôle et la surveillance des ressources** utilisées ou attribuées aux processus.

Ressources contrôlables :

- Mémoire
- Processeurs (CPUs)
- Périphériques (Devices)
- Exécution (CPU time, scheduling)
- Paquets réseau envoyés
- Nombre de processus dans un cgroup
- Etc.

Namespaces = isolation (ce que le conteneur *voit*)

cgroups = limitation (ce que le conteneur *peut consommer*)

Interface avec le noyau : cgroupfs

Les cgroups sont exposés par le noyau à travers le pseudo-système de fichiers :

```
/sys/fs/cgroup/      <- cgroupfs
```

- Le suivi des ressources et les limitations sont implantés dans un ensemble de **sous-systèmes** (un par type de ressource)
- Un **cgroup** est une collection de processus restreints par un ensemble de limites/-paramètres définis à travers le cgroupfs
- Un cgroup est associé à des **contrôleurs de ressources**, définis comme des attributs dans les fichiers et sous-répertoires de cgroupfs

Note : l'arborescence exacte et le nom des fichiers peuvent varier d'une version noyau à une autre.

Contrôleurs disponibles

Lister les contrôleurs compilés dans le noyau :

```
cat /proc/cgroups
```

#subsys_name	hierarchy	num_cgroups	enabled
cpuset	4	1	1
cpu	8	1	1
cpuacct	8	1	1
blkio	6	1	1
memory	3	1	1
devices	10	84	1
freezer	7	1	1
net_cls	9	1	1
perf_event	5	1	1
net_prio	9	1	1
hugetlb	0	1	0
pids	2	1	1

Exemple : création d'un cgroup

```
cd /sys/fs/cgroup/cpuset
mkdir cg1          # créer un nouveau cgroup
ls cg1
```

Fichiers générés automatiquement dans cg1/ :

cgroup.clone_children	cpuset.memory_pressure
cgroup.procs	cpuset.memory_spread_page
cpuset.cpu_exclusive	cpuset.memory_spread_slab
cpuset.cpus	cpuset.mems
cpuset.effective_cpus	cpuset.sched_load_balance
cpuset.effective_mems	cpuset.sched_relax_domain_level

`cgroup.procs` : liste des PIDs des processus appartenant à ce cgroup, donc contrôlés par ce contrôleur (ici `cpuset`).

Lire les CPUs autorisés pour ce cgroup :

```
cat cg1/cpuset.cpus
0-4
```

Les processus dont le PID est dans `cgroup.procs` ne pourront s'exécuter que sur les CPUs 0 à 4.

Logiciels de conteneurisation

LXC – Linux Containers

- Site : <https://linuxcontainers.org/>
- Documentation : <https://linuxcontainers.org/lxc/documentation/>
- Basé sur les **namespaces** pour l'isolation entre conteneurs
- Lorsqu'une application est lancée dans un conteneur LXC, **plusieurs espaces de noms sont créés** pour cette application afin d'isoler le conteneur

Docker

- Site : <https://www.docker.com/>
- Documentation : <https://docs.docker.com/get-started/overview/#the-underlying-techn>
- Basé sur les **namespaces** pour l'isolation entre conteneurs
- Lorsqu'une application est lancée, via les daemons **containerd** et la commande **runc** (*low-level container runtime*), plusieurs espaces de noms sont créés

Architecture Docker

Composants Docker

- **dockerd** : démon de management (haut niveau)
- **containerd** : démon d'exécution (runtime), via **containerd-shim**
- **runc** : outil bas niveau de création et d'exécution des conteneurs (basé sur les namespaces et cgroups)

Chaîne d'appels lors du lancement d'un conteneur :

`docker run` → `dockerd` → `containerd` → `containerd-shim` → `runc` → conteneur

Récapitulatif : support noyau pour la conteneurisation

Mécanisme noyau	Rôle
Namespaces	Isolation : chaque conteneur a sa propre vue du système (réseau, PID, fichiers, utilisateurs...)
cgroups	Limitation et surveillance des ressources : CPU, mémoire, débit réseau, nombre de processus...
Système de fichiers en couches	Images légères (overlayfs, UnionFS) : partage des couches communes entre conteneurs

Sources

- `man cgroups` – Manuel Linux
- LXC Documentation : <https://linuxcontainers.org/lxc/documentation/>
- Docker Overview : <https://docs.docker.com/get-started/overview/>
- Copyright Naceur.Malouch@lip6.fr – All rights reserved.